Speed control code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

int motorPin1 = 5;
int motorPin2 = 6;
int enablePin = 9;

int zonePin0 = A0;
int zonePin1 = A1;
int zonePin2 = A2;
int zonePin3 = A3;

int vehicleSpeed = 0;  // *Actual speed*
int cutoffSpeed = 0;   // *Speed limit*
int zoneType = 0;

const int SCHOOL_ZONE_SPEED = 30;
const int HOSPITAL_ZONE_SPEED = 50;
const int TRAFFIC_ZONE_SPEED = 70;
const int HIGHWAY_ZONE_SPEED = 100;
const int HIGHWAY_ZONE_LIMIT = 120;

void setup() {
  lcd.begin();
  lcd.backlight();
  Serial.begin(9600);

  pinMode(motorPin1, OUTPUT);
  pinMode(motorPin2, OUTPUT);
  pinMode(enablePin, OUTPUT);

  pinMode(zonePin0, INPUT);
  pinMode(zonePin1, INPUT);
  pinMode(zonePin2, INPUT);
  pinMode(zonePin3, INPUT);

  lcd.setCursor(0, 0);
  lcd.print("Vehicle Init...");
```

```
  delay(2000);
  lcd.clear();
}

// *Function to control motor speed*
void setMotorSpeed(int speed) {
  analogWrite(enablePin, speed);
  digitalWrite(motorPin1, HIGH);
  digitalWrite(motorPin2, LOW);
  vehicleSpeed = speed;  // *Update actual speed*
}

void loop() {
  if (analogRead(zonePin0) > 400) {
    zoneType = 1;
  } else if (analogRead(zonePin1) > 400) {
    zoneType = 2;
  } else if (analogRead(zonePin2) > 400) {
    zoneType = 3;
  } else if (analogRead(zonePin3) > 400) {
    zoneType = 4;
  } else {
    zoneType = 0;
  }

  switch (zoneType) {
    case 1:
      cutoffSpeed = SCHOOL_ZONE_SPEED;
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("School Zone");
      setMotorSpeed(SCHOOL_ZONE_SPEED);
      break;
    case 2:
      cutoffSpeed = HOSPITAL_ZONE_SPEED;
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Hospital Zone");
      setMotorSpeed(HOSPITAL_ZONE_SPEED);
      break;
    case 3:
      cutoffSpeed = TRAFFIC_ZONE_SPEED;
      lcd.clear();
      lcd.setCursor(0, 0);
```

```
      lcd.print("Traffic Zone");
      setMotorSpeed(TRAFFIC_ZONE_SPEED);
      break;
    case 4:
      cutoffSpeed = HIGHWAY_ZONE_LIMIT;
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Highway Zone");
      setMotorSpeed(HIGHWAY_ZONE_SPEED);
      break;
    default:
      cutoffSpeed = -1;
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Normal Mode");
      setMotorSpeed(120);
      break;
  }

  // *✅ Display actual speed and speed limit correctly*
  lcd.setCursor(0, 1);
  lcd.print("Speed:");
  lcd.print(vehicleSpeed);

  if (cutoffSpeed != -1) { // Display limit only if in a zone
    lcd.print(" Lmt:");
    lcd.print(cutoffSpeed);
  }

  Serial.print("Zone Type: ");
  Serial.print(zoneType);
  Serial.print(" | Speed: ");
  Serial.print(vehicleSpeed);
  Serial.print(" | Limit: ");
  Serial.println(cutoffSpeed != -1 ? String(cutoffSpeed) : "-");

  delay(1000);
}
```

V2v code

```
#include <esp_now.h>
#include <WiFi.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Define pin numbers
#define UP_PIN 14
#define DOWN_PIN 27
#define LEFT_PIN 26
#define RIGHT_PIN 25

// Maximum number of devices
#define MAX_DEVICES 5

// Universal MAC Address (Broadcast)
uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

// Data structure for device data
typedef struct struct_message {
  char color[10];
  char numberPlate[15];
  int direction; // 0-Stop, 1-Up, 2-Down, 3-Left, 4-Right
} struct_message;

// Create instances for sending and receiving data
struct_message myData; // For controller's data
struct_message deviceData[MAX_DEVICES]; // Array to hold data for multiple devices
int deviceCount = 0; // Track connected devices

// LCD instance (I2C address 0x27, 16 columns, 2 rows)
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Direction descriptions
const char *directions[] = {"S", "F", "B", "L", "R"};

// Callback when data is sent
void OnDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
  Serial.print("Send Status: ");
  Serial.println(status == ESP_NOW_SEND_SUCCESS ? "Success" : "Fail");
}
```

```
// Function to find or add a device by color
int findOrAddDevice(const char *color) {
  // Check if the device already exists
  for (int i = 0; i < deviceCount; i++) {
    if (strcmp(deviceData[i].color, color) == 0) {
      return i; // Return index if found
    }
  }

  // Add a new device if it doesn't exist
  if (deviceCount < MAX_DEVICES) {
    strcpy(deviceData[deviceCount].color, color);
    return deviceCount++;
  }

  return -1; // No space to add a new device
}

// Callback when data is received
void OnDataRecv(const uint8_t *mac, const uint8_t *incomingData, int len) {
  struct_message incomingReadings;
  memcpy(&incomingReadings, incomingData, sizeof(incomingReadings));

  Serial.printf("Received Data - Color: %s, Number Plate: %s, Direction: %d\n",
                incomingReadings.color, incomingReadings.numberPlate,
incomingReadings.direction);

  // Find or add the device
  int index = findOrAddDevice(incomingReadings.color);
  if (index != -1) {
    // Update the corresponding device data
    strcpy(deviceData[index].numberPlate, incomingReadings.numberPlate);
    deviceData[index].direction = incomingReadings.direction;

    // Update the LCD
    updateLCD();
  }
}

// Function to update LCD display
void updateLCD() {
  lcd.clear(); // Clear LCD

  // Display information for each connected device
```

```
  for (int i = 0; i < deviceCount; i++) {
    lcd.setCursor(0, i); // Adjust for the row
    lcd.print(deviceData[i].color);
    lcd.print(": ");
    lcd.print(deviceData[i].numberPlate);
    lcd.print(" ");
    lcd.print(directions[deviceData[i].direction]);
  }
}

// Reset connection status if no data is received
void resetConnections() {
  deviceCount = 0;
  lcd.clear();
}

void setup() {
  Serial.begin(115200);

  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.print("Initializing...");

  pinMode(UP_PIN, INPUT_PULLDOWN);
  pinMode(DOWN_PIN, INPUT_PULLDOWN);
  pinMode(LEFT_PIN, INPUT_PULLDOWN);
  pinMode(RIGHT_PIN, INPUT_PULLDOWN);

  WiFi.mode(WIFI_STA);

  if (esp_now_init() != ESP_OK) {
    Serial.println("Error initializing ESP-NOW");
    lcd.setCursor(0, 1);
    lcd.print("ESP-NOW Failed");
    return;
  }

  esp_now_register_send_cb(OnDataSent);
  esp_now_register_recv_cb(OnDataRecv);

  esp_now_peer_info_t peerInfo = {};
  memcpy(peerInfo.peer_addr, broadcastAddress, 6);
  peerInfo.channel = 0;
```

```
  peerInfo.encrypt = false;

  if (esp_now_add_peer(&peerInfo) != ESP_OK) {
    Serial.println("Failed to add peer");
    lcd.setCursor(0, 1);
    lcd.print("Peer Add Failed");
    return;
  }

  Serial.println("ESP-NOW Initialized");
  lcd.clear();
  lcd.print("   WELCOME SIR");

  // Initialize  data
  strcpy(myData.color, "B");
  strcpy(myData.numberPlate, "TN01AB333");  // Example number plate
  myData.direction = 0; // Initial direction is stop
}

void loop() {
  if (digitalRead(UP_PIN) == HIGH) {
    myData.direction = 1; // Forward
  } else if (digitalRead(DOWN_PIN) == HIGH) {
    myData.direction = 2; // Backward
  } else if (digitalRead(LEFT_PIN) == HIGH) {
    myData.direction = 3; // Left
  } else if (digitalRead(RIGHT_PIN) == HIGH) {
    myData.direction = 4; // Right
  } else {
    myData.direction = 0; // Stop
  }

  esp_err_t result = esp_now_send(broadcastAddress, (uint8_t *)&myData, sizeof(myData));
  if (result == ESP_OK) {
    Serial.println("data sent successfully");
  } else {
    Serial.println("Error sending data");
  }

  static unsigned long lastReceiveTime = millis();
  if (millis() - lastReceiveTime > 5000) {
    resetConnections();
    lastReceiveTime = millis();
  }
```

```
}
```

Radar

```
#include <Servo.h>

// Define pins
#define TRIG_PIN 9   // Ultrasonic sensor TRIG
#define ECHO_PIN 10  // Ultrasonic sensor ECHO (changed from 11)
#define BUZZER_PIN 5 // Buzzer connected to pin 5
#define OBJECT_DISTANCE 30 // Distance threshold for object detection (cm)

Servo servo;
int servoPin = 11; // Servo control pin (unchanged)

void setup() {
  servo.attach(servoPin);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  servo.write(90); // Initialize servo at the center (90°)
  Serial.begin(9600); // For debugging
}

void loop() {
  // Radar functionality
  for (int angle = 0; angle <= 180; angle += 10) {
    radarSweep(angle);
  }
  for (int angle = 180; angle >= 0; angle -= 10) {
    radarSweep(angle);
  }
}

void radarSweep(int angle) {
  servo.write(angle);
  delay(20); // Allow servo to stabilize
```

```
  // Measure distance
  long distance = measureDistance();

  if (distance > 0 && distance <= OBJECT_DISTANCE) {
    tone(BUZZER_PIN, 1000); // Shout while object is detected
  } else {
    noTone(BUZZER_PIN); // Stop shouting when no object is in range
  }

  // Debugging: print distance
  Serial.print("Servo Angle: ");
  Serial.print(angle);
  Serial.print(" | Distance: ");
  Serial.println(distance);
}

long measureDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH);
  long distance = duration * 0.034 / 2; // Convert to cm

  // Discard invalid readings
  if (distance <= 0 || distance > 400) { // Ultrasonic range: 2-400 cm
    return -1; // Invalid distance
  }
  return distance;
}
```

Rain

```
// Define pins
#define RAIN_SENSOR_PIN A0 // Rain sensor signal pin
#define BUZZER_PIN 5       // Buzzer pin
#define RAIN_THRESHOLD 300 // Threshold for rain detection

void setup() {
  pinMode(RAIN_SENSOR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  Serial.begin(9600); // For debugging
}

void loop() {
  int rainValue = analogRead(RAIN_SENSOR_PIN); // Read rain sensor value

  // Debugging: Print the rain sensor value
  Serial.print("Rain Sensor Value: ");
  Serial.println(rainValue);

  if (rainValue < RAIN_THRESHOLD) {
    // Rain detected: Activate buzzer with lower frequency
    tone(BUZZER_PIN, 500); // Frequency: 500 Hz (lower pitch)
    delay(500);            // Buzzer ON duration
    noTone(BUZZER_PIN);    // Turn off the buzzer
    delay(500);            // Pause before next cycle
  } else {
    noTone(BUZZER_PIN); // No rain: Ensure the buzzer is off
  }

  delay(1000); // Check rain sensor every second
}
```

Radar+tft+rain+speaker with connection

🔌 **Pin Map (matches your last wiring plan)**

- **TFT ILI9341 (SPI):** `CS=10`, `DC=9`, `RESET=8`, `MOSI=11`, `MISO=12`, `SCK=13`, `VCC=5V`, `GND=GND`, `LED=3.3V or 5V via 100Ω`
- **Servo (sweep):** `D6` (signal), `5V`, `GND` *(use separate 5V supply if servo is noisy; share GND)*
- **HC-SR04:** `TRIG=D4`, `ECHO=D5`, `VCC=5V`, `GND=GND`
- **YL-83 (module AO):** `A0`, `5V`, `GND`
- **DFPlayer Mini:** `TX → D2` (Arduino RX), `RX → D3` (Arduino TX via ~1kΩ), `VCC=5V`, `GND=GND`, **Speaker → SPK1 & SPK2**

```
/*
  Mini Radar + Rain Voice Alerts (Arduino UNO)

  Hardware (pins):
  TFT ILI9341 SPI: CS=10, DC=9, RESET=8, MOSI=11, MISO=12, SCK=13, VCC=5V, GND=GND
  Servo: D6 (signal), 5V, GND
  HC-SR04: TRIG=D4, ECHO=D5, VCC=5V, GND=GND
  YL-83 AO: A0
  DFPlayer Mini: TX -> D2 (Arduino RX), RX -> D3 (Arduino TX via ~1k), VCC=5V, GND=GND, Speaker -> SPK1/SPK2

  Audio files (microSD):
  /MP3/0001.mp3  Object close
  /MP3/0002.mp3  Medium rain
  /MP3/0003.mp3  Heavy rain
  /MP3/0004.mp3  Alarming rain

  Notes:
  - DFPlayer MINI cannot change playback speed programmatically; put pre-recorded files at desired speeds.
  - Use a separate 5V supply for the servo if display flickers; share grounds.
*/
```

```
#include <Servo.h>
#include <SPI.h>
#include "Ucglib.h"
#include "SoftwareSerial.h"
#include "DFRobotDFPlayerMini.h"

// ---------------- Pins ----------------
#define TRIG_PIN   4
#define ECHO_PIN   5
#define SERVO_PIN  6
#define RAIN_PIN   A0

// TFT on hardware SPI, control pins:
Ucglib_ILI9341_18x240x320_HWSPI ucg(/*cd=*/ 9, /*cs=*/ 10, /*reset=*/ 8);

// Servo and Radar geometry
Servo baseServo;
const int Ymax = 240;
const int Xmax = 320;
const int Xcent = Xmax / 2;
const int baseLine = 210;
const int scanline = 185;

// DFPlayer
SoftwareSerial mp3Serial(2, 3); // RX (to DFPlayer TX), TX (to DFPlayer RX via ~1k)
DFRobotDFPlayerMini mp3;

// ------------- Behavior tuning -------------
const int OBJECT_ALERT_CM = 30;       // distance threshold for object alert
const unsigned long OBJ_ALERT_COOLDOWN = 2500UL;   // ms, avoid spamming
const unsigned long RAIN_CHECK_PERIOD  = 2000UL;   // ms between analog reads (normal)
const unsigned long RAIN_COOLDOWN_MED  = 30000UL;  // ms between repeats of same
level
const unsigned long RAIN_COOLDOWN_HIGH = 20000UL;  // ms
const unsigned long RAIN_COOLDOWN_ALRM = 10000UL;  // ms

// YL-83 thresholds (tweak per your module; lower = wetter)
const int RAIN_THR_NONE    = 800;  // >800: dry/very light -> silent
const int RAIN_THR_MEDIUM  = 500;  // 500..800: medium
const int RAIN_THR_HIGH    = 300;  // 300..500: high
// <300: alarming

// Audio files (in /MP3/)
const uint16_t FILE_OBJECT  = 1;
```

```
const uint16_t FILE_RAIN_MED= 2;
const uint16_t FILE_RAIN_HI = 3;
const uint16_t FILE_RAIN_AL = 4;

// Volumes per level (0..30)
const uint8_t VOL_MED = 22;
const uint8_t VOL_HI  = 26;
const uint8_t VOL_AL  = 28;
const uint8_t VOL_OBJ = 25;

// Timing state
unsigned long lastObjAlert   = 0;
unsigned long lastRainCheck  = 0;
unsigned long lastMedAnnounce= 0;
unsigned long lastHiAnnounce = 0;
unsigned long lastAlAnnounce = 0;

// ---------------- Utility ----------------
void tftBoot() {
  ucg.begin(UCG_FONT_MODE_SOLID);
  ucg.setRotate90();

  ucg.setFontMode(UCG_FONT_MODE_TRANSPARENT);
  ucg.setColor(0, 0, 100, 0);
  ucg.setColor(1, 0, 100, 0);
  ucg.setColor(2, 20, 20, 20);
  ucg.setColor(3, 20, 20, 20);
  ucg.drawGradientBox(0, 0, 320, 240);

  ucg.setColor(0, 255, 0);
  ucg.setPrintPos(70, 120);
  ucg.setFont(ucg_font_logisoso32_tf);
  ucg.print("Mini Radar");

  delay(600);
  // clear screen
  ucg.setColor(0, 0, 0, 0);
  for (int s = 0; s < 240; s++) {
    ucg.drawHLine(0, s, 320);
    delay(1);
  }
  ucg.setFontMode(UCG_FONT_MODE_SOLID);
  ucg.setFont(ucg_font_helvR08_hr);
}
```

```
void drawScaleText() {
  ucg.setColor(0, 180, 0);
  ucg.setPrintPos(144, 44);  ucg.print("1.00");
  ucg.setPrintPos(144, 100); ucg.print("0.60");
  ucg.setPrintPos(144, 165); ucg.print("0.30");
}

void drawRadarBase() {
  ucg.setColor(0, 180, 0);
  ucg.drawDisc(Xcent, baseLine + 1, 3, UCG_DRAW_ALL);
  ucg.drawCircle(Xcent, baseLine + 1, 210, UCG_DRAW_UPPER_LEFT);
  ucg.drawCircle(Xcent, baseLine + 1, 210, UCG_DRAW_UPPER_RIGHT);
  ucg.drawCircle(Xcent, baseLine + 1, 135, UCG_DRAW_UPPER_LEFT);
  ucg.drawCircle(Xcent, baseLine + 1, 135, UCG_DRAW_UPPER_RIGHT);
  ucg.drawCircle(Xcent, baseLine + 1, 70,  UCG_DRAW_UPPER_LEFT);
  ucg.drawCircle(Xcent, baseLine + 1, 70,  UCG_DRAW_UPPER_RIGHT);
  ucg.drawLine(0, baseLine + 1, Xmax, baseLine + 1);
}

void clsFast() {
  ucg.setColor(0, 0, 0, 0);
  for (int s = 0; s < 240; s++) {
    ucg.drawHLine(0, s, 320);
    // keep it snappy; avoid extra delay here
  }
}

// HC-SR04 distance in cm
int readDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 30000UL); // timeout ~30ms
  if (duration == 0) return 999; // out of range
  return (int)(duration * 0.034f / 2.0f);
}

// ---------- Audio helpers ----------
void playMp3(uint16_t file, uint8_t vol) {
  mp3.volume(vol);
  // files must be /MP3/000X.mp3
```

```
  mp3.playMp3Folder(file);
}

void tryObjectAlert() {
  unsigned long now = millis();
  if (now - lastObjAlert >= OBJ_ALERT_COOLDOWN) {
    playMp3(FILE_OBJECT, VOL_OBJ);
    lastObjAlert = now;
  }
}

void checkRainAndAnnounce() {
  unsigned long now = millis();
  if (now - lastRainCheck < RAIN_CHECK_PERIOD) return;
  lastRainCheck = now;

  int rainVal = analogRead(RAIN_PIN);
  // Debug
  // Serial.print("Rain A0: "); Serial.println(rainVal);

  if (rainVal > RAIN_THR_NONE) {
    // Dry / very light — say nothing
    return;
  } else if (rainVal > RAIN_THR_MEDIUM) {
    if (now - lastMedAnnounce >= RAIN_COOLDOWN_MED) {
      playMp3(FILE_RAIN_MED, VOL_MED);
      lastMedAnnounce = now;
    }
  } else if (rainVal > RAIN_THR_HIGH) {
    if (now - lastHiAnnounce >= RAIN_COOLDOWN_HIGH) {
      playMp3(FILE_RAIN_HI, VOL_HI);
      lastHiAnnounce = now;
    }
  } else {
    if (now - lastAlAnnounce >= RAIN_COOLDOWN_ALRM) {
      playMp3(FILE_RAIN_AL, VOL_AL);
      lastAlAnnounce = now;
    }
  }
}

// ---------- Setup / Loop ----------
void setup() {
  pinMode(TRIG_PIN, OUTPUT);
```

```
  pinMode(ECHO_PIN, INPUT);
  pinMode(RAIN_PIN, INPUT);

  baseServo.attach(SERVO_PIN);
  baseServo.write(90);

  // Serial for debug
  Serial.begin(115200);

  // DFPlayer init
  mp3Serial.begin(9600);
  if (!mp3.begin(mp3Serial)) {
    Serial.println("DFPlayer Mini not found! Check wiring/SD.");
    // continue without blocking; radar can still run
  } else {
    mp3.EQ(DFPLAYER_EQ_NORMAL);
    mp3.volume(25); // default
  }

  // TFT boot screen
  tftBoot();
}

void loop() {
  // Sweep Right-to-Left
  drawRadarBase();
  drawScaleText();

  for (int deg = 180; deg > 4; deg -= 2) {
    baseServo.write(deg);

    // sweeping beam (3 fades)
    int f = deg - 4;
    ucg.setColor(0, 255, 0);
    ucg.drawLine(Xcent, baseLine, scanline * cos(radians(f)) + Xcent, baseLine - scanline *
sin(radians(f)));
    f += 2;
    ucg.setColor(0, 128, 0);
    ucg.drawLine(Xcent, baseLine, scanline * cos(radians(f)) + Xcent, baseLine - scanline *
sin(radians(f)));
    f += 2;
    ucg.setColor(0, 0, 0);
    ucg.drawLine(Xcent, baseLine, scanline * cos(radians(f)) + Xcent, baseLine - scanline *
sin(radians(f)));
```

```
    // Distance + alerts
    int d = readDistanceCm();
    if (d < OBJECT_ALERT_CM) tryObjectAlert();

    // Cursor dot
    if (d < 100) {
      ucg.setColor(255, 0, 0);
      ucg.drawDisc(2.2 * d * cos(radians(deg)) + Xcent, -2.2 * d * sin(radians(deg)) + baseLine, 1,
UCG_DRAW_ALL);
    } else {
      ucg.setColor(255, 255, 0);
      ucg.drawDisc(208 * cos(radians(deg)) + Xcent, -208 * sin(radians(deg)) + baseLine, 1,
UCG_DRAW_ALL);
    }

    // HUD
    ucg.setColor(255, 255, 0);
    ucg.setPrintPos(20, 230);  ucg.print("DEG: ");
    ucg.setPrintPos(54, 230);  ucg.print(deg);
    ucg.setPrintPos(240, 230); ucg.print(d); ucg.print(" cm   ");

    // Background rain check (non-blocking)
    checkRainAndAnnounce();
  }

  clsFast();

  // Sweep Left-to-Right
  drawRadarBase();
  drawScaleText();

  for (int deg = 1; deg < 176; deg += 2) {
    baseServo.write(deg);

    int f = deg + 4;
    ucg.setColor(0, 255, 0);
    ucg.drawLine(Xcent, baseLine, scanline * cos(radians(f)) + Xcent, baseLine - scanline *
sin(radians(f)));
    f -= 2;
    ucg.setColor(0, 128, 0);
    ucg.drawLine(Xcent, baseLine, scanline * cos(radians(f)) + Xcent, baseLine - scanline *
sin(radians(f)));
    f -= 2;
```

```
    ucg.setColor(0, 0, 0);
    ucg.drawLine(Xcent, baseLine, scanline * cos(radians(f)) + Xcent, baseLine - scanline *
sin(radians(f)));

    int d = readDistanceCm();
    if (d < OBJECT_ALERT_CM) tryObjectAlert();

    if (d < 100) {
      ucg.setColor(255, 0, 0);
      ucg.drawDisc(2.2 * d * cos(radians(deg)) + Xcent, -2.2 * d * sin(radians(deg)) + baseLine, 1,
UCG_DRAW_ALL);
    } else {
      ucg.setColor(255, 255, 0);
      ucg.drawDisc(208 * cos(radians(deg)) + Xcent, -208 * sin(radians(deg)) + baseLine, 1,
UCG_DRAW_ALL);
    }

    ucg.setColor(255, 255, 0);
    ucg.setPrintPos(20, 230);  ucg.print("DEG: ");
    ucg.setPrintPos(54, 230);  ucg.print(deg);
    ucg.setPrintPos(240, 230); ucg.print(d); ucg.print(" cm    ");

    checkRainAndAnnounce();
  }

  clsFast();
}
```